Bitwise NOT: →

NOT

0	1
1	0

$5 \rightarrow 0101$
$\sim 5 \rightarrow 1010 = 10$

$\boxed{\sim 5\ != 10 = -6}$

$-6 = 10$

Capgemini
Oracle

$5-6 \rightarrow 200$
$20 \rightarrow$ 1st
$10 \rightarrow$ 2nd

5-6	4

$H \rightarrow C$
$C \rightarrow H$

Compiler: How! $\sim 5 = -6$ 10

1 − 0
0 − 1] Reciprocal

(-6) absolute value = value $= 6$

[−6 = 10] Theoretical

Binary = 0110

MCQ → ~5
a) 2 b) 3
c) −6 d) 4

2's Com { 1's com = 1001
+1 = 0001
1010 =

$\boxed{n \rightarrow \sim n = -n-1}$ Complement

$-5-1 = -6$

$-49 \quad \sim(-49) = -(-49)-1 = 49-1 = 48$

[Dry Run:] arr = [~~1~~, ~~2~~, ~~3~~, ~~6~~, ~~3~~, ~~2~~, ~~1~~], 7 arrays

res = 0

8 4 2 1
0000
0001

0001
0010

0011
0011

0000
0110

0110

0110
0011

0101
0010

0111
0001

0110 = 6

Decision Making, Looping, Branching, Jumps:

① Conditional Statements:

a) Simple if statement → 1 condition
b) if-else " → 2 conditions
c) if-else-if else ladder → more than 2 conditions (ugly 7,8)
d) Nested if statement → Branching [Condition inside Condition]
e) Switch Case → Better version of (c)
f) Ternary Operator → Short-hand if-else

DRY → don't Repeat Yourself.

HW [ch → A−Z, vowel / consonant]
[a, e, i, o, u] V
[others] consonants

Looping Statements:

* Repetitive Tasks: Iterations

(i) while ✓
(ii) do while ✓
(iii) for ✓
(iv) for-each / enhanced for loop

CS/IS

Syntax

While Loop Syntax:

```
while (condition) {
    Statements;
    increment/decrement;
}
```

True

enter

Entry Controlled Loop

Do-while loop syntax:

```
do {
    Statements;
    increment/decrement;
} while (condition);
```

True − continue
False − exit

Exit Controlled Loop

At least once the code is executed. Then condition is checked & the statements are executed based on it.

The for-loop syntax:

```
for (initialization; condition; inc/dec ) {
    Statements;
}
```

```
for ( int i=0; i<=10; i++) {
    printf ("%d", i);
}
```

condition inc

[Difference b/w for & while]

* steps

(while) uncertain infinite

(for) certain finite

* Take input from the user a number 'n' = 10; Display even/odd till that number 1−10 but without using "%".

if (n%2==0) even
odd

$1/2 = 0$

AND

0101
0001

0001

2 | 5
4
--
1

$5/2 = 2$ int

Bitwise AND
$(n \& 1) \rightarrow 1 \rightarrow T$
$(0101 \& 0001)$ 0 → F

4+1 = 5

	8 4 2 1	
0	0000	→ E
1	0001	→ O
2	0010	→ E
3	0011	→ O
4	0100	→ E
5	0101	→ O

e+1 = odd

* Find the sum of 1st n natural numbers:

(i) using for-loop.
(ii) using while-loop.
(iii) using formula!

55 ← 10

$= \left[\frac{n \times (n+1)}{2}\right] = \frac{\cancel{10}^{5} \times 11}{\cancel{2}} = 55.$

* Nested-Loops:

Loop inside Loop

```
for (          ){
    for (          ){
    }
}
```

i = 1 to 10
j = 1 to 10

Multiplication Table

rows
1
2
3
4
5
10

1 → 1x1=1 1x2=2 - - - - 1x10=10
2 →
10 → 10x1=10 - - - 10x10=100

Loop Variables | Iteration Variables

```
#include <stdio.h>
int main() {
    for (int i=1; i<=10; i++){
        for (int j=1; j<=10; j++){
            printf(format: "%dx%d=%d\t", j,i,i*j);
        }
        printf(format: "\n");
    }
    return 0;
}
```

i = 1, 2, 3

j → 1 to 10

j → 1 to 10

Outer

innermost

for{ → 3
for{ → 2
for{ → 1